Machine Control System

System Design

Central machine state management

Servo station · Drive hardware station

ctrlX CORE PLC (IEC 61131-3 Structured Text)

Draft for review

9 October 2026

Generated from the design documents in `design/`. Those Markdown files remain the source of truth.

Contents

1 Overview **3**

1.1 Documents and status . . . 3
1.2 System structure . . . 3
1.3 Implementation order . . . 4
1.4 Rules every implementation follows . . . 4
1.5 Still to confirm . . . 4

2 Central Machine State Management **5**

2.1 Purpose and scope . . . 5
2.2 States . . . 5
2.3 Transitions . . . 6
2.4 Jumps: abort and timeouts . . . 7
2.5 Clients: interface, client FB and wrapper . . . 8
2.5.1 `I_StateClient` . . . 8
2.5.2 `FB_StateRequest` (wrapper) . . . 9
2.6 The manager: how clients are combined . . . 10
2.7 Implementation notes . . . 11
2.8 Assumptions and decisions . . . 11

3 Unit: Servo Station **13**

3.1 Purpose and scope . . . 13
3.2 Axes . . . 13
3.3 `FB_ServoAxisMgr`: power, faults, reset . . . 13
3.3.1 Behaviour per machine state . . . 13
3.3.2 Fault detection: abort requests . . . 14
3.3.3 Reset . . . 15
3.3.4 Internal structure . . . 15
3.4 `FB_ServoMotionSeq`: motion sequence . . . 15
3.4.1 Internal states (`E_ServoSeqState`) . . . 15
3.4.2 Behaviour notes . . . 16
3.4.3 Echo and done per machine state . . . 17
3.5 Unit interface . . . 17
3.6 Constants . . . 18
3.7 Decisions and assumptions . . . 18

4 Unit: Drive Hardware Station **20**

4.1 Purpose and scope 20

4.2 Drive status word: bits 15/14 and bit 13 20

4.3 Behaviour per machine state 21

4.4 Ordering with the servo station, via hardware status only 21

4.5 Fault detection: abort requests 22

4.6 Reset 22

4.7 Interface 23

4.8 Internal structure 23

4.9 Decisions and assumptions 23

1 Overview

This document describes the control design of the machine: one central state manager that owns the machine's operating state, and the units that follow it. The design is **library-agnostic on purpose**. It says *what* each block commands. The motion and I/O libraries are chosen when the code is written.

1.1 Documents and status

Part	Covers	Status
Central state management (Ch. 2)	`FB_StateManager`, the broadcast machine state, `I_StateClient`, `FB_StateClient`, `FB_StateRequest`, echo and done, abort jumps	No open questions. 4 assumptions to confirm.
Servo station (Ch. 3)	`FB_ServoAxisMgr` (power, faults, reset, torque-free at the end of Aborting), `FB_ServoMotionSeq` (position, gear-in, VM speed, hold, stop, abort)	No open questions. 4 assumptions to confirm.
Drive hardware station (Ch. 4)	`FB_DriveSupplyMgr`: DC bus supply on/off from drive status bits 15/14, bit-13 fault aborts, reset	Complete

1.2 System structure

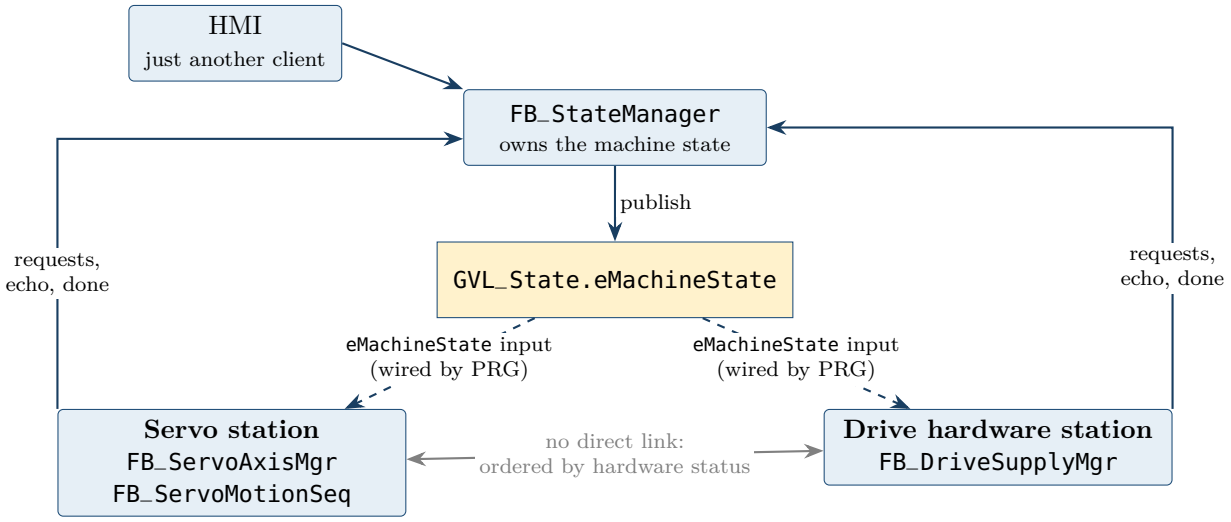


Figure 1.1: Requests go up through the clients. The state comes down through the broadcast, which the calling program wires into each unit's `eMachineState` input.

1.3 Implementation order

1. **Central state management:** `E_MachineState`, `I_StateClient`, `FB_StateClient`, `FB_StateRequest`, `FB_StateManager` and `GVL_State`. Everything else depends on these.
2. **Drive hardware station:** `FB_DriveSupplyMgr`. It has no motion and relies only on status words, so it's the simplest unit to bring up first.
3. **Servo station:** `FB_ServoAxisMgr`, then `FB_ServoMotionSeq`.
4. **Wiring:** a program that calls the manager and the units, assigns client indexes (`CLIENT_*` constants) and maps the hardware I/O (status words, supply signals, axis references).

1.4 Rules every implementation follows

These come from the house style:

- **Sequences use four sections:** jumps → transitions (`CASE`, transitions only) → on change (`IF _eState <> _eLastState`: common actions, then entry actions) → cyclic (sub-FB calls, then all outputs last: command values as expressions of the state and copies of internal values).
- **No rising edges.** Commands are levels. A terminal state holds until its command is removed.
- **A POU never reads its own outputs.** The logic works on internal _ variables, and the outputs are copies written at the end.
- **No first-scan logic.** Every state machine starts in its default (Off/Idle) state, which does nothing.
- **Enums vs constants:** an enum when the state is an output, `VAR CONSTANT` steps when it's internal. Implicit numbering, no gaps.
- **Any fault aborts.** Fault info is kept from the first cause and cleared on entering Idle.
- **Units never talk to each other.** They follow the broadcast state, report through their client, and order themselves by hardware status (e.g. drive status words).
- **No global data in an FB.** An FB never reads or writes a global variable. Global constants are allowed. The machine state reaches each unit through its `eMachineState` input, wired by the calling program, so another layer of state management can be added later without changing the units.

1.5 Still to confirm

- The four assumptions of the central state management (§2.8) and the four of the servo station (§3.7).
- The motion library has been chosen for implementation. How it provides the gear-in slave offset and the on-the-fly VM speed is settled in the code.

2 Central Machine State Management

2.1 Purpose and scope

One central **state manager**, `FB_StateManager`, owns the operating state of the whole machine. Each unit (axis group, conveyor, heater, station, HMI…) has an `FB_StateClient` attached and talks to the manager only through it, using the standard `FB_StateRequest` wrapper (§2.5).

- **The state is broadcast.** The manager publishes one value, `GVL_State.eMachineState`. Each unit receives it as an `eMachineState` **input**, which the calling program wires to the broadcast. Units never read the GVL themselves (open question 7).
- **The client carries only requests upward.** The unit uses it to echo the state it's acting on, report done, and make requests (abort, hold, …).

The manager reads all clients each scan, decides the state and publishes it. Units never talk to the manager or to each other directly.

The state manager owns:

- the machine state and every transition between states;
- the rules for which requests act in which states;
- timeouts on the transitional states;
- recording the first abort cause and which client raised it.

It does not own:

- *how* a unit turns on, starts, stops, holds or aborts. That logic lives in the unit.
- Safety functions. E-stop, guard and STO circuits are handled by the safety system. The safety status reaches the manager only as an abort request, from a client owned by the safety-monitoring unit.

2.2 States

State	Kind	Meaning	What the units do
Off	Stable (default)	The machine is de-energised. This is the power-up state.	Everything is off and safe.
TurningOn	Transitional	The machine is bringing systems up.	Power drives, enable heaters, establish comms…
Idle	Stable	Everything is on and ready, but no production. There's no fault in Idle.	Hold ready. No motion.
Starting	Transitional	Preparing for production.	Home or move to start positions, ramp up, prime…
Running	Stable	Producing.	Normal cyclic operation.
Stopping	Transitional	Controlled stop at the end of a cycle.	Finish the current cycle and bring to rest.
Holding	Transitional	Pausing production in a resumable way.	Pause at a safe point and keep position and context.

State	Kind	Meaning	What the units do
Hold	Stable	Paused, and production can resume.	Stay paused.
UnHolding	Transitional	Resuming from Hold.	Restore motion and context.
Aborting	Transitional	Fast stop because something is wrong.	Stop as quickly as is safe, in the shortest way.
Aborted	Stable	Stopped after an abort and waiting for reset.	Stay stopped and safe.
Resetting	Transitional	Recovering from an abort.	Clear faults, re-enable, return to a known condition.

Every **transitional** state has a timeout (§2.4) and leaves when **all clients are done** with it. **Stable** states have no timeout, and they leave on requests.

2.3 Transitions

`AllDone` means every active client reports that it has finished the current state (§2.6).

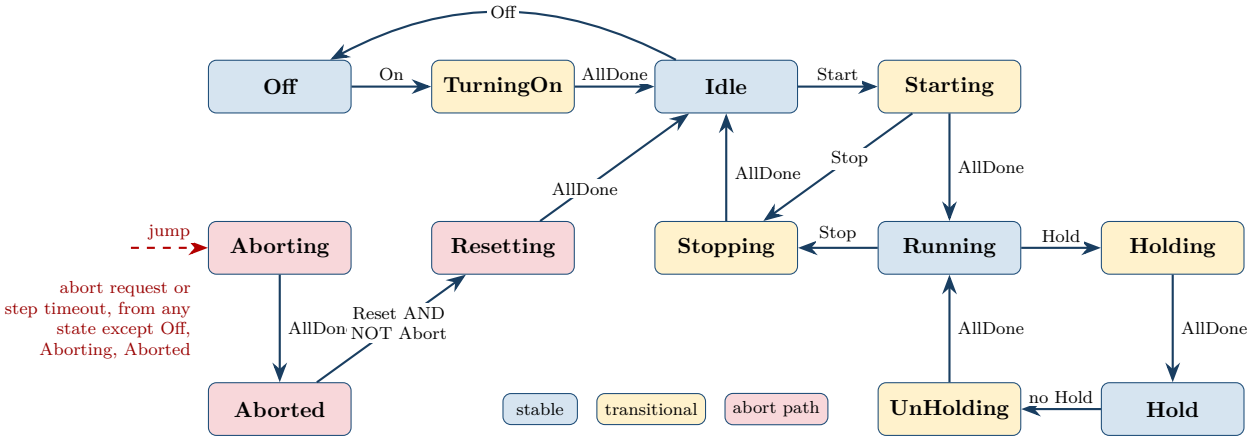


Figure 2.1: Machine states. Requests are the combined (OR) requests of all clients, as levels.

Transition table (normal transitions)

From	Condition	To
Off	`On AND NOT Off`	TurningOn
TurningOn	`AllDone`	Idle
Idle	`Off`	Off
Idle	`Start AND NOT Stop AND NOT Off`	Starting
Starting	`Stop`	Stopping
Starting	`AllDone`	Running
Running	`Stop`	Stopping
Running	`Hold`	Holding
Stopping	`AllDone`	Idle
Holding	`AllDone`	Hold
Hold	`NOT Hold`	UnHolding
UnHolding	`AllDone`	Running
Aborting	`AllDone`	Aborted
Aborted	`Reset AND NOT Abort`	Resetting
Resetting	`AllDone`	Idle

- **Priorities inside Running:** stop is checked before hold, so if both are requested the machine stops.
- **Stop during Starting** always goes to Stopping, whether or not starting has completed. Stop is checked before `AllDone`.
- **Hold lasts while any client requests it.** The machine resumes only when **no** client requests hold any more, so a starved infeed and an operator pause can overlap safely.
- **Off is reached only from Idle.** Going from Running to Off therefore passes through Stopping first.
- **Aborted → Resetting requires that no client still requests an abort.** A held abort can't be reset away.

2.4 Jumps: abort and timeouts

The jumps are evaluated **before** the transitions, in priority order, from every state except Off, Aborting and Aborted:

Prio.	Condition	To	Recorded
1	Any client requests an abort	Aborting	That client's abort code, the client's index, and the state it came from
2	Step timer of the current transitional state has expired	Aborting	"Timeout", the state, the first client that wasn't done, and whether it never acknowledged or never finished (§2.6)

The transition `CASE` then runs as normal on the state the jump set. Exits from Aborting are themselves conditions (`AllDone`), so a transition that fires in the same scan is a correct one.

Fault information:

- Only the **first** cause is recorded. The jumps aren't evaluated in Aborting or Aborted, so later faults can't overwrite it.
- **Units don't raise fault-based abort requests in Aborting, Aborted or Resetting.** A fault is reset *in* Resetting, but Aborted → Resetting requires that no client requests an abort, so a fault that's still present would block its own reset. A fault that survives the reset leaves the unit not done, and the Resetting timeout aborts again. Deliberate aborts (operator, safety) keep requesting, so they still block the reset.
- The cause is **cleared on entry to Idle**. Idle means nothing is wrong, so there's no fault to show. The cause stays visible for the whole Aborting → Aborted → Resetting path.

Timeout in Aborting. If clients don't confirm Aborting within the abort timeout, the machine still goes to Aborted. The client that didn't confirm is recorded as a second diagnostic. Reaching a resettable state matters more than waiting forever.

2.5 Clients: interface, client FB and wrapper

A client has three standard layers. They're the same for every unit.

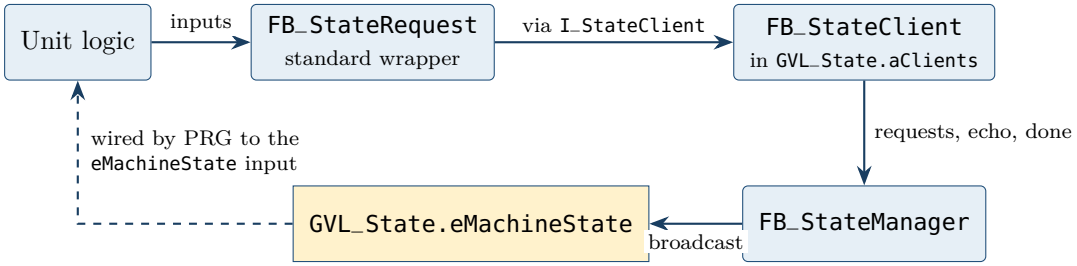


Figure 2.2: The three client layers, and the state coming back down.

1. **`I_StateClient` (interface).** The standard way to talk to a client: request levels, state echo and done, all going up to the manager.
2. **`FB_StateClient IMPLEMENTS I_StateClient`.** The actual client. It stores the request levels, echo and done. The manager reads these instances (§2.6). The client doesn't carry the state; that's the broadcast.
3. **`FB_StateRequest` (wrapper).** The one block a unit uses. It holds the attached `I_StateClient`, checks it isn't null, and forwards the unit's inputs through the interface every call. Units never touch the interface or the client directly, so using a client takes a single FB call.

2.5.1 `I_StateClient`

Member	Kind	Meaning
`Request(bOn, bOff, bStart, bStop, bHold, bAbort, uiAbortCode, bReset)`	Method	Sets this client's request levels. Every call overwrites all of them.

`Report(eStateEcho, bDone, bEnable)`	Method	The state the unit is acting on, whether it's finished, and whether the client takes part at all.

2.5.2 `FB_StateRequest` (wrapper)

Inputs	Outputs
`itfClient : I_StateClient`: the attached client	`bAttached`: `itfClient <> 0`
`bOn`, `bOff`, `bStart`, `bStop`, `bHold`, `bReset`: requests (levels, default FALSE)	
`bAbort`, `uiAbortCode`: abort request and its reason	
`eStateEcho`, `bDone`: progress on the current state	
`bEnable` (default TRUE): FALSE switches the unit out of the machine state	

A unit connects only the inputs it uses, and the rest stay FALSE. It's called with the unit's other sub-FBs at the end, before the unit's outputs:

```
_fbStateReq(
    itfClient   := itfClient,                  // wired by the caller to
    ↪ aClients[CLIENT_INFEED]
    bAbort      := _eState = E_InfeedState.Faulted,
    uiAbortCode := _uiFaultCode,
    bHold       := _eState = E_InfeedState.Starved,
    eStateEcho  := _eMachineStateSeen,         // machine state this unit has taken
    ↪ over
    bDone       := _bStepDone);
```

Inside, the wrapper does:

```
IF itfClient <> 0 THEN itfClient.Request(...); itfClient.Report(...); END_IF
```

All requests are written every call, so releasing an input releases the request.

The unit's own state machine follows the machine state it's given: its transitions read its `eMachineState` input (latched once per scan into an internal variable), and when it has taken a machine state over it echoes that state back through the wrapper. The calling program wires the input, normally `eMachineState := GVL_State.eMachineState`.

Done is the echoed state plus a done bit

The manager counts a client as done only when `eStateEcho = current state AND bDone`.

- **Echo without done:** the unit has picked up the new state and is working on it. The manager can show which units are still busy, and a timeout can name the one that never acknowledged.
- **Echo of an old state:** the unit hasn't taken the new state over yet. Its `bDone` refers to the old state and is ignored. A stale done from Holding can never satisfy UnHolding.
- The unit sets both from its own state machine: `eStateEcho` follows the state it's actually in, and `bDone` comes from its own logic. Both are levels, and there's nothing to clear.

Units with nothing to do in a state echo it and set `bDone` straight away. For example, a heater has nothing to hold, so in Holding it reports done immediately.

Commands are levels. The HMI is just another client, so operator buttons are held requests too. There are no rising edges anywhere: stable states wait for a request to change, so a held request can't re-trigger anything. That works the same way as `Done`, which waits for `Execute` to drop.

2.6 The manager: how clients are combined

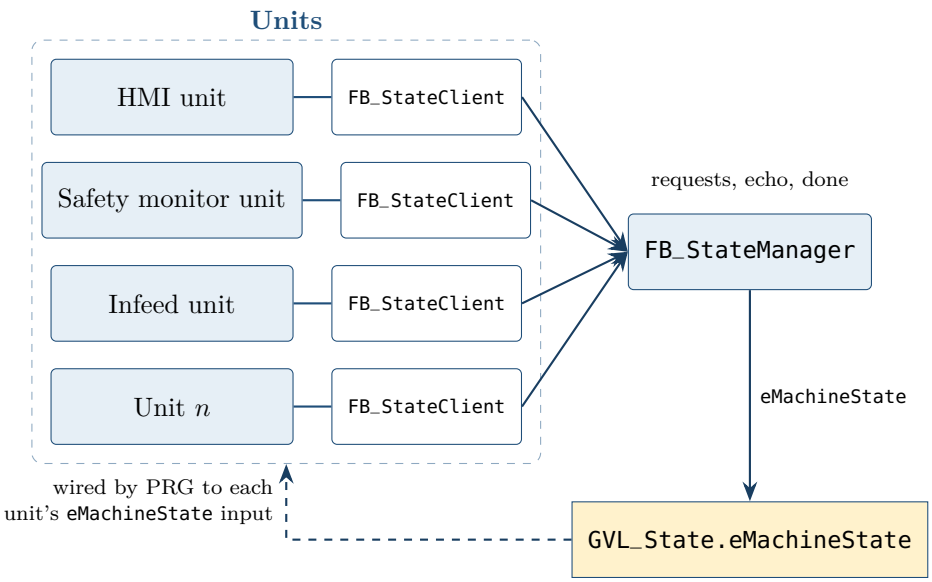


Figure 2.3: The manager combines every client, then publishes one state.

Each scan, `FB_StateManager` runs the house sequence structure, with one extra section in front:

0. **Read clients.** A `FOR` loop over the client array builds the combined requests and `AllDone`:
   - each request (`Abort`, `Hold`, `Stop`, `Start`, `On`, `Off`, `Reset`) is the OR across all active clients;
   - `AllDone` is the AND of `client.eStateEcho = _eState AND client.bDone` across all active clients;
   - the loop records the **first** client requesting an abort (index and code), and the **first** client that isn't done. It also notes whether that client hasn't acknowledged (echo $\neq$ state) or is still working (echo = state, not done), for timeout diagnostics.
1. **Jumps:** abort, then timeout (§2.4).

2. **Transitions:** a `CASE` with only the state changes in Figure 2.1, plus `ELSE` → Aborting.
3. **On change:** `IF _eState <> _eLastState`. First the common actions: restart the step timer and clear the timeout. Then the per-state entry actions: set the timeout for each transitional state, and clear the abort info on entering Idle.
4. **Cyclic:** the manager's outputs, including the broadcast `eMachineState`, are copied from internal variables, and the step timer is called last.

The clients are read at the start and the state is published at the end, so units always see the state the manager settled on in this scan. A unit that reports done is picked up in the manager's next scan. Data flows one way through each path: requests go up through the clients, and the state comes down through the broadcast.

2.7 Implementation notes

Registering clients. There were two options (open question 1):

- **A. Array passed in:** `VAR_IN_OUT aClients : ARRAY[*] OF FB_StateClient`, with the array declared in a GVL (`GVL_State.aClients[1..MAX_CLIENTS]`). Each unit is given a fixed index (constant), e.g. `GVL_State.aClients[CLIENT_INFEED]`. Bounds come from `LOWER_BOUND`/`UPPER_BOUND`. This is simple and visible online, and adding a unit means adding one constant. **Decided**.
- **B. Registration by reference:** each unit's client calls `fbManager.Register(THIS^)` once, and the manager stores `REFERENCE TO FB_StateClient` in a fixed-size table. Clients live inside their units, but it adds init-order and online-change concerns.

Active clients. A client counts toward `AllDone` and the request ORs only while it's enabled. **A client starts disabled**, and it becomes enabled when a unit's `FB_StateRequest` reports `bEnable` (default TRUE). Unused slots in `aClients` are therefore ignored, so they can't block `AllDone`. A unit that's switched out (e.g. an optional station) passes `bEnable := FALSE` instead of reporting done everywhere.

Types and conventions:

- `E_MachineState` is an enum with `{attribute 'qualified_only'}` and `{attribute 'strict'}`, numbered implicitly from `Off := 0`. It's an enum because the state is exposed.
- **Internal state:** `_eState`, `_eLastState`, `_uiAbortCode`, `_iAbortClient` and `_eAbortFrom` are the manager's internal variables. The outputs are copies.
- **Timeouts and client indexes** are `VAR CONSTANT`s.
- **No first-scan logic.** The machine starts in Off, which does nothing, and every client's defaults (`eStateEcho` = Off, `bDone` = FALSE) are consistent with that.

Manager outputs, published to `GVL_State`: `eMachineState` (the broadcast), plus `uiAbortCode`, `iAbortClient`, `eAbortFrom` and `iNotDoneClient` for the HMI and diagnostics. A unit that wants to know whether it caused the abort compares `iAbortClient` with its own client index.

2.8 Assumptions and decisions

Assumed (please confirm):

1. "someStartCmd → Start → Starting" has been read as a single **Starting** state. There's no separate **Start** state.
2. Reset leads to **Idle**: Aborted → Resetting → Idle.
3. Abort applies from every state except Off, Aborting and Aborted.
4. A transitional-state timeout counts as an abort, not as a separate Faulted state.

Open questions, all Decided:

1. **Client registration:** option A. `GVL_State.aClients[1..MAX_CLIENTS]`, an array of `FB_StateClient`, is passed to the manager as `VAR_IN_OUT ARRAY[*]`, and each unit uses a `CLIENT_*` index constant.
2. **Request policy:** all requests are equal. Every enabled client's requests are combined (OR) the same way, whichever client they come from.
3. **Stop while in Hold or Holding:** not needed. Stop acts from Running only, so a held machine resumes first. (The servo sequence would handle a stop from any step anyway.)
4. **Stop while Starting:** a stop request in Starting always goes to Stopping, whether or not starting has completed. Units must handle a stop from mid-start; the servo sequence does this with its stop jump.
5. **A TurningOff state:** no. Going to Off just powers the axes down. The drive supply already waits until no axis is under torque before it switches off, so the order is right without an extra state.
6. **Start held through a stop:** the machine just starts again. A start request still held when Stopping returns to Idle restarts it, as written.
7. **How units get the broadcast state:** as an input. No FB reads or writes a global variable (global constants are fine). Every unit has an `eMachineState : E_MachineState` input, and only the calling program wires it to `GVL_State.eMachineState`. If the machine grows and needs another layer of state management (e.g. a sub-manager per section of the line), a unit can be given that sub-manager's state instead of the machine state, without any change inside the unit. It also makes every unit's dependencies visible at its call site.

3 Unit: Servo Station

3.1 Purpose and scope

The servo station has **two real servo axes** and **one virtual master (VM)**. In production, both real axes are geared 1:1 to the VM at caller-supplied phase offsets, and the line speed is the VM's velocity.

The unit follows the machine state it's given as its `eMachineState` input (wired by the calling program to the broadcast `GVL_State.eMachineState`) and is split into two blocks. Each block has its own state client (`FB_StateRequest`):

Block	Job
`FB_ServoAxisMgr`	Axis power on/off, fault detection with abort requests, and axis fault reset.
`FB_ServoMotionSeq`	All motion: positioning, gear-in, VM speed, hold/unhold, stop and abort sequences.

The manager waits for **both** clients before it leaves a transitional state. Power and motion therefore stay independent, and neither block needs to know the other's internals.

This chapter says *what* each block commands. It doesn't commit to a motion library. Commands are named by function (power, reset, absolute move, gear-in, velocity move, halt, stop), and the implementation picks the library that provides them.

3.2 Axes

Name	Type	Role
`Ax1`	Real servo, rotary modulo 360°	Slave 1, geared 1:1 to the VM at `lrAx1Offset`
`Ax2`	Real servo, rotary modulo 360°	Slave 2, geared 1:1 to the VM at `lrAx2Offset`
`VM`	Virtual axis, rotary modulo 360°	Master. Its velocity is the line speed.

The offsets are **inputs from the caller** (§3.5). For the current machine the caller passes `OFFSET` for Ax1 and `360 - OFFSET` for Ax2. The blocks don't know or assume that relationship.

3.3 `FB_ServoAxisMgr`: power, faults, reset

3.3.1 Behaviour per machine state

Machine state	Axis power	Done when
Off	Off	All 3 axes report disabled
TurningOn	On, once the axis reports ready for power (see below)	All 3 powered, no axis error
Idle, Starting, Running, Stopping, Holding, Hold, UnHolding	On	Immediately: nothing to do beyond staying powered
Aborting	On while the motion block stops the axes. Once **all axes are at rest** (standstill, stopped, error-stop or disabled), they're commanded **torque-free** while the machine is still in Aborting.	All 3 report disabled (torque-free)
Aborted	Off	All 3 report disabled
Resetting	Reset both axes on entry, then on once the axes report ready for power	All 3 powered, no axis error

Power is a level derived from the machine state and the axis states:

```
bWantPower   := eMachineState <> Off AND eMachineState <> Aborted
                AND NOT (eMachineState = Aborting AND bAllAxesAtRest);
bPowerEnable := bWantPower AND bPowerReady;
```

- `bAllAxesAtRest`: every axis is in standstill, stopped, error-stop or disabled. "Disabled" is included, so once the axes are torque-free they stay torque-free for the rest of Aborting, with no latch needed.
- `bPowerReady`, per axis: the axis's own **ready-for-power feedback** from the motion system ("the drive is ready for power up"). For a real axis it only comes once the drive hardware station has the DC bus up, so the ordering still comes from the hardware status, with no unit-to-unit link. The servo station deliberately **doesn't read the drive status word**; the drive hardware station owns it. Because of that, any axis can be switched to a virtual axis for testing without changing this unit. **Decided** (it replaces the earlier "drive status word bit 15" condition).
- **Torque-free at the end of Aborting** is what lets the drive hardware station confirm that no axis is under torque and shut the supply down within Aborting (Chapter 4).
- If the motion block's stop commands report errors because power was removed, those are ignored: the machine is already in Aborting, and the first abort cause has been recorded.

3.3.2 Fault detection: abort requests

Any fault aborts. Outside Aborting, Aborted and Resetting (§2.4), the block requests an abort (through its client) while any of these is TRUE:

Abort code	Condition
`ABORT_AXIS_ERROR + i`	Axis i is in its error-stop state or has a drive error
`ABORT_POWER_LOST + i`	Axis i should be powered (`bPowerEnable` has been TRUE, outside TurningOn and Resetting) but isn't

The first matching code is reported. The central manager keeps only the first abort machine-wide.

3.3.3 Reset

On **entry to Resetting**, a reset is issued to **both real axes**, whatever caused the abort. Any fault aborts, so Resetting always follows a fault, and the reset doesn't need to know which one. The axes are torque-free (disabled) at that point, which is one of the states where the reset is allowed.

- The reset of Ax1 (drive 0) also clears the drive supply, which is built into drive 0.
- After the reset, power comes back on as soon as the axes report ready for power.
- The block reports done once no axis has an error and all are powered.
- If an axis won't reset, the manager's Resetting timeout aborts again, with this client recorded as the one not done.

3.3.4 Internal structure

This block has no sequence of its own. Everything is a level function of the machine state and the axis status, so there's no internal state machine. It's written as:

1. read the axis status (results from the previous scan);
2. fault conditions → abort request and code;
3. echo and done from the machine state;
4. command levels (power enable), and the reset of both axes on entry to Resetting;
5. call the sub-FBs (power, reset and status for each axis), then copy the outputs last.

3.4 `FB_ServoMotionSeq`: motion sequence

3.4.1 Internal states (`E_ServoSeqState`)

State	Commands (levels from this state)	Leaves when
Idle	None. All axes are at standstill and ungeared.	The machine is Starting
Positioning	Absolute moves, all in parallel and by the shortest way: Ax1 → `lrAx1Offset`, Ax2 → `lrAx2Offset`, VM → 0	All three done → GearingIn
GearingIn	Gear Ax1 and Ax2 to the VM in **absolute position mode** against the VM's **set point**, ratio 1:1, with slave offsets `lrAx1Offset` / `lrAx2Offset`, synchronising by the shortest way	Both in sync → Synced

State	Commands (levels from this state)	Leaves when
Synced	Gearing stays engaged. VM at standstill.	The machine is Running or UnHolding → VmRunning. (Stopping is a jump.)
VmRunning	VM velocity move at `lrSpeedCmd`, **updated on the fly**	The machine is Holding → VmHalting; speed command changed → VmSpeedChange. (Stopping is a jump.)
VmSpeedChange	One scan, used when the motion library can't update a running velocity move (the case for the chosen library). Its entry latches the new `lrSpeedCmd`.	Next scan → VmRunning, whose entry re-issues the velocity move at the new speed
VmHalting	Halt the VM at the normal deceleration	VM stopped: if the machine is Holding → Synced, if Stopping → Ungearing
Ungearing	Halt Ax1 and Ax2. This ends the gearing, and the slaves are already at zero speed.	Both at standstill → Idle
AbortVm	Stop the VM at the **abort deceleration**	VM stopped, or VM in error → AbortSlaves
AbortSlaves	Stop Ax1 and Ax2 at the abort deceleration, which ends the gearing	Both stopped or in error → Idle

Jumps (house structure), abort first, then stop:

- If the machine state is **Aborting** and the block isn't already in `AbortVm`, `AbortSlaves` or `Idle`, it jumps to `AbortVm`. That one rule covers an abort from any motion state: during positioning, gearing, running or halting. Any motion command error raises an abort request through this block's client, with the command as the abort code. **Any fault aborts.**
- If the machine state is **Stopping** and the block isn't in `Idle`, `VmHalting`, `Ungearing` or an abort step, it jumps to `VmHalting` (halt the VM, then ungear). The sequence therefore handles a stop from any step, so the manager could allow stop from Starting or Hold without servo changes.

All motion commands are issued **once, on entry to their state**, and tracked afterwards; leaving a state re-arms the command for next time.

3.4.2 Behaviour notes

- **Gearing is absolute, to the VM's set point.** The slave's command is the VM's set point $\times 1$ + the slave offset, so the gearing *enforces* the phase. Positioning has already put the slaves at their offsets with the VM at 0, so gear-in needs no correction move. If a slave is ever out of phase, it's pulled back the shortest way.
- **Speed changes apply on the fly** in VmRunning, including during UnHolding, without stopping the VM. Where the library can't update a running velocity command, the one-scan

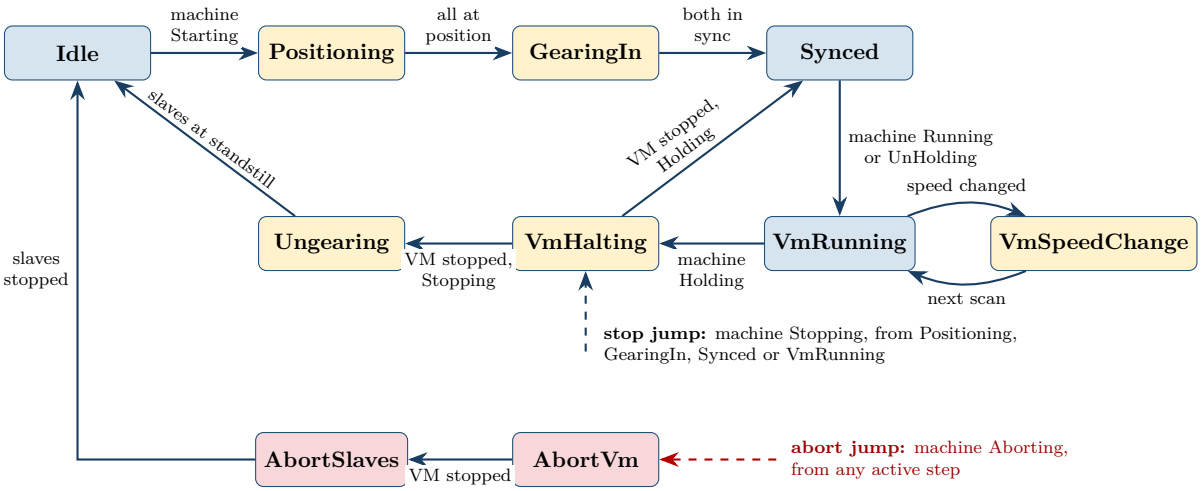


Figure 3.1: Motion sequence. Dashed: jumps evaluated before the transitions.

`VmSpeedChange` state re-triggers it through the state machine. No edges are needed. The command must be one that keeps the VM moving while it's re-triggered.

- **Abort deceleration** (`ABORT_DEC`) is separate from the normal VM deceleration (`VM_DEC`). While geared, the slaves follow the VM down at whichever deceleration the VM is using.
- **A fault while running:** the drive's own error reaction stops the faulted axis. The machine aborts, AbortVm brings the VM down at `ABORT_DEC`, and the healthy slave follows it down while geared. AbortSlaves then ends the gearing. A faulted axis counts as stopped, so the abort can't hang on it.

3.4.3 Echo and done per machine state

Machine state	Done when the sequence is in
Starting	`Synced`
Holding	`Synced` (VM stopped, still geared)
UnHolding	`VmRunning` and the VM at the commanded speed. If the speed changes mid-UnHolding, done drops until the new speed is reached.
Stopping	`Idle` (VM stopped, gearing ended, all at standstill)
Aborting	`Idle` (VM stopped, gearing aborted, all stopped)
Off, TurningOn, Idle, Running, Hold, Aborted, Resetting	Immediately: no motion work

The block echoes the machine state on the scan after its own transition logic has seen it. The manager ignores done while the echo is still the old state (§2.5).

3.5 Unit interface

Input	Meaning

`eMachineState`	The machine state to follow, wired by the calling program
`lrSpeedCmd`	VM velocity, in VM units/s. It can change at any time. **Limiting it is the caller's job**, so the unit doesn't clamp it.
`lrAx1Offset`, `lrAx2Offset`	Slave phase offsets from the VM, used both as the positioning targets and as the gear-in slave offsets. They're supplied by the caller.
Axis references	`Ax1`, `Ax2`, `VM`, of whatever axis reference type the chosen library uses
Axis cyclic status	Per axis (Ax1, Ax2, VM): ready for power, powered, standstill, error, in sync, data valid, from the motion system. The drive status words are **not** inputs; the drive hardware station owns them.
Client attachments	Two `I_StateClient`s, one per block, e.g. `GVL_State.aClients[CLIENT_SERVO_AXES]` and `[CLIENT_SERVO_MOTION]`

Outputs (diagnostics, copied from internal variables): `eSeqState`, the per-axis status summary, and the abort code this unit raised.

3.6 Constants

Constant	Meaning
`POS_VEL`, `POS_ACC`, `POS_DEC`	Dynamics for the start positioning moves
`VM_ACC`, `VM_DEC`	Normal VM ramp for run, hold and stop
`ABORT_DEC`	Abort deceleration for the VM and slaves. It's separate from `VM_DEC`.
`ABORT_AXIS_ERROR`, `ABORT_POWER_LOST`, `ABORT_MOTION_CMD`	Base abort codes. The axis or command index is added to them.

The machine-level timeouts for TurningOn, Starting, Stopping and the others live in the central manager. They must allow for the longest positioning move.

3.7 Decisions and assumptions

Decided:

1. Positioning takes the **shortest way**.
2. Gearing is **absolute, to the VM's set point**, with the slave offsets applied by the gearing itself.
3. Abort uses its **own deceleration**, separate from the normal ramp.
4. **Speed changes on the fly**, in Running and UnHolding.
5. **Any fault aborts.** There are no partial or local stops.
6. **The caller limits the speed command.** The unit doesn't clamp it.
7. **The caller supplies the slave offsets.**
8. **The design is library-agnostic.** The motion library is chosen at implementation time.

9. **Torque-free at the end of Aborting.** Once all axes are at rest they're commanded torque-free while still in Aborting, so the drive supply can shut down before Aborted.
10. **Power readiness comes from the axis's ready-for-power feedback**, not the drive status word, so any axis can be switched to a virtual axis for testing.

Assumed (please confirm):

1. The axes use absolute encoders, so no homing is needed before positioning in Starting.
2. All three axes are modulo 360°.
3. In Hold, the slaves stay geared, and only the VM stops.
4. The offsets don't change while geared. If they can, the gear-in offset has to be one the library can update live.

4 Unit: Drive Hardware Station

4.1 Purpose and scope

The drive hardware station manages the **drive power supply**, the DC bus that feeds the servo drives. It's one block, `FB_DriveSupplyMgr`, with one state client (`FB_StateRequest`). It follows the machine state it's given as its `eMachineState` input, wired by the calling program to the broadcast `GVL_State.eMachineState`.

It owns:

- switching the supply on, but only once every drive reports it's ready for power;
- switching the supply off, but only once no axis is under torque;
- detecting a supply that doesn't come on, drops out or faults, and requesting an abort for it;
- waiting in Resetting for supply and drive faults to clear. The reset itself is the drive reset of drive 0, which hosts the supply, issued by the servo station.

It does not own: axis power or torque. That belongs to the servo station's axis manager. The two units never talk to each other. This unit reads the drive status words, and the servo station reads its axes' ready-for-power feedback from the motion system. That's enough to order them correctly (§4.4).

As with the other units, this design is library-agnostic. The supply command, supply feedback and drive status words are plain signals, mapped from whatever I/O the hardware provides.

4.2 Drive status word: bits 15/14 and bit 13

Every axis drive reports its readiness in bits 15 and 14 of its status word:

Bit 15	Bit 14	Meaning	Used as
0	0	Not ready for power	Blocks power-on
0	1	Ready for power	**Ready for power** (bit 14 or bit 15 set)
1	0	Power section ready, torque-free	Ready for power, not under torque
1	1	Under torque	**Under torque** (bit 14 and bit 15 set)

Two derived levels, taken across all axis drives:

- `bAllReadyForPower := every drive has (bit 14 OR bit 15)`
- `bAnyUnderTorque := any drive has (bit 14 AND bit 15)`

Bit 13 is the fault bit. It's checked on every axis drive's status word and on the **supply's own status word**:

- `bAnyDriveFault := any drive has bit 13`
- `bSupplyFault := supply status word bit 13`

4.3 Behaviour per machine state

Machine state	Supply	Done when
Off	Off, once no axis is under torque	Supply off
TurningOn	On, once all drives are ready for power	Supply on
Idle, Starting, Running, Stopping, Holding, Hold, UnHolding	On	Immediately: nothing to do beyond staying on
Aborting	Stays on until **no axis is under torque**, then off	No axis under torque **and** supply off
Aborted	Off	Immediately
Resetting	Reset supply and drive faults, then on once all drives are ready for power	No faults **and** supply on

The supply command as one level rule

```
bWantOn     := eMachineState <> Off AND eMachineState <> Aborted
               AND eMachineState <> Aborting;
// switch on only when all drives are ready, then stay on ...
_bSupplyCmd := (bWantOn AND (_bSupplyCmd OR bAllReadyForPower))
// ... and never switch off while any axis is under torque
               OR (_bSupplyCmd AND bAnyUnderTorque);
```

- **Ready check, at switch-on only.** Once the supply is on, the drives move to "power section ready" or "under torque". Both still count as ready, and the supply holds itself on.
- **Torque check, at switch-off.** This applies to any switch-off, whether to Off or in Aborting. The supply goes off only after every axis is torque-free.
- `_bSupplyCmd` is internal. Reading its value from the previous scan is a self-hold, not a read of an output.

4.4 Ordering with the servo station, via hardware status only

Phase	Drive hardware station	Servo axis manager
TurningOn / Resetting	Waits until all drives are *ready for power*, then switches the supply on	Waits until each axis reports *ready for power* (motion feedback, which follows the DC bus for a real axis), then enables the axes
Aborting	Holds the supply on while any axis is *under torque*, then switches off	Stops the axes, then commands them **torque-free** once at rest, still in Aborting
Off	Switches off once no axis is under torque	Disables the axes

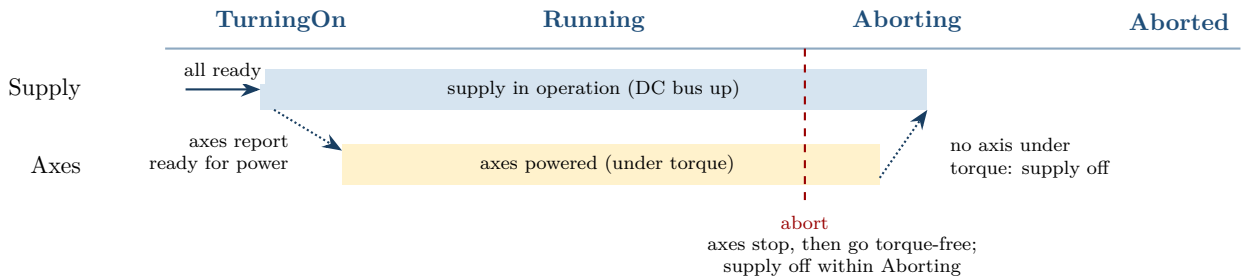


Figure 4.1: The units order themselves by hardware status alone. The supply comes up before the axes power on, and goes down only after every axis is torque-free.

The machine can only leave **Aborting** when both units are done: the axes are torque-free and the supply is off. **Aborted** is therefore reached with the bus already down.

4.5 Fault detection: abort requests

Any fault aborts. Outside Aborting, Aborted and Resetting (§2.4), the block requests an abort (through its client) while any of these is TRUE:

Abort code	Condition
`ABORT_SUPPLY_CTRL`	The supply didn't reach the commanded mode in time (or its controller rejected the command)
`ABORT_SUPPLY_FAULT`	The supply reports an internal error, read from its status word. **This also covers a supply that drops out after it was on.**
`ABORT_DRIVE_FAULT + i`	Drive i fault: drive i status word bit 13

Only power-on aborts on time. The supply controller supervises every mode change, on and off. An off change only happens in Off, where the manager ignores aborts, or in Aborting, where fault aborts aren't raised. So in practice only a supply that fails to come on causes an abort. Once the supply is up, a drop-out shows as a supply fault.

There's no local timeout for drives that never become ready for power. The supply simply isn't commanded, this client never reports done, and the manager's TurningOn or Resetting timeout aborts with this client recorded as not done.

4.6 Reset

This block doesn't reset anything itself. The supply is built into drive 0, which is Ax1's drive, so the **drive reset of drive 0** clears supply errors as well as drive errors. The servo station's axis manager resets both axes on entry to Resetting, whatever the fault was. Any fault aborts, so Resetting always follows a fault, and the reset doesn't need to know which one.

- This block waits in Resetting. When the faults clear, the drives report ready for power and the supply rule brings the supply back to operation.
- If the supply controller itself is in error, its enable is held off while Resetting, which clears it.
- The block is done when there are no faults and the supply is in operation.
- If the reset doesn't clear things, the manager's Resetting timeout aborts again.

4.7 Interface

Input	Meaning
`eMachineState`	The machine state to follow, wired by the calling program
`awDriveStatus`	Status words of the **drives in use** on this supply (array: drive 0 = Ax1, drive 1 = Ax2): bits 15/14 readiness, bit 13 fault. The spare drive 2 isn't included.
`wSupplyStatus`	Supply unit status word: operating mode and internal error
Client attachment	One `I_StateClient`, e.g. `GVL_State.aClients[CLIENT_DRIVE_HW]`
Output	**Meaning**
`wSupplyCtrl`	Supply unit control word (in/out): drives the supply to operation or mains off
`awDriveDcBusCtrl`	DC bus power control of each drive (in/out): "DC bus power" is set while the supply is in operation
Diagnostics	`bSupplyInOperation`, `bSupplyOff`, `bAllReadyForPower`, `bAnyUnderTorque`, `bAnyDriveFault`, `bSupplyFault`, the abort code raised

Constants:

- `SUPPLY_MODE_TIMEOUT`: the longest a supply mode change may take
- `ABORT_SUPPLY_CTRL`, `ABORT_SUPPLY_FAULT`, `ABORT_DRIVE_FAULT`: abort codes

4.8 Internal structure

As with the servo axis manager, there's no internal state machine. Everything is a level function of the machine state and the hardware status:

1. read the status: decode each drive's status word with a sub-function (ready for power, under torque, fault), and read the supply's mode and error from its controller;
2. fault conditions → abort request and code;
3. echo and done from the machine state;
4. command levels: `_bSupplyCmd` → supply target mode (operation / mains off), the supply controller's enable, and DC bus power to each drive;
5. call the sub-FBs (supply controller, client wrapper), then copy the outputs last.

4.9 Decisions and assumptions

Decided:

1. Power-on is allowed only when every drive is **ready for power** (status bit 14 or 15).
2. A supply that doesn't power on means **abort**. **Only power-on aborts on time**. A drop-out after power-on is caught as a supply fault.

3. In Aborting, the supply goes off only after **no axis is under torque** (bits 14 and 15 not both set), and that happens **within Aborting**. The servo station makes its axes torque-free at the end of Aborting.
4. Resetting **brings the supply back on** once the faults are cleared. **The reset is the drive reset of drive 0** (the supply is built into it), issued by the servo axis manager. This block has no reset output.
5. **A supply or drive fault means abort**: a supply internal error, or bit 13 of any drive status word.
6. **The supply is off in Off and Aborted, and on in every other state**, including Idle.
7. **There are 3 drives and 1 supply.** Drive 0 is Ax1 and hosts the supply, and drive 1 is Ax2. These two get DC bus power while the supply is in operation. **Drive 2 is a spare and doesn't count**: it isn't part of readiness, torque or fault checks.
8. **One supply feeds all 3 drives** (confirmed). The VM has no drive.
9. **A drop-out is caught by the supply fault** (bit 13). Only power-on is timed.